JOB BOARD

Как работает твой Java-проект

Подробный учебный разбор для junior Java developer

От нажатия кнопки в браузере до записи в базе данных. Каждый слой разобран на конкретных классах, методах и запросах из проекта.

Проект: zinchenko-job-board Java 21 • Spring Boot 3.5.16 • Spring Data JPA • H2 Подготовлено для Yehor Zinchenko • 27 октября 2026

Читай рядом с IntelliJ: открывай упомянутый файл, находи метод и проверяй объяснение на работающем приложении. Код в приложении к документу - снимок текущей версии, а не будущих изменений.

Материал подготовлен с помощью Codex. Здесь отдельно отмечены существующее поведение, упрощения учебного проекта и упражнения, которые пока не реализованы.

Содержание

01. Что именно мы построили 4

02. Карта файлов и порядок чтения 5

03. Что делает main и как стартует Spring 6

04. Spring создаёт зависимости за нас 7

05. Полный путь открытия страницы 8

06. HTTP, JSON и адреса API 9

07. DTO: что такое JobRequest 10

08. Валидация: где отсекаются ошибки 11

09. Entity: как Job превращается в запись 12

10. Что хранит база и как сопоставляются поля 13

11. Repository: интерфейс без своей реализации 14

12. Service: зачем отдельный слой 15

13. Создание: от кнопки до commit 16

14. Обновление и удаление по ID 17

15. @Transactional без магических обещаний 18

16. Контроллер переводит HTTP в Java 19

17. Ошибки и ApiExceptionHandler 20

18. H2 и application.properties 21

19. HTML и CSS: устройство интерфейса 22

20. JavaScript: запросы и безопасный вывод 23

21. JavaScript: форма, Edit и Delete 24

22. Maven: что именно делает pom.xml 25

23. Ошибка release version 5: что произошло 26

24. Как устроены интеграционные тесты 27

25. Что проверяет каждый из семи тестов 28

26. Как разбираться с ошибкой по шагам 29

27. Учебное изменение: фильтр по городу 30

28. Ответы на собеседовании: основные вопросы 31

29. Вопросы глубже и честные ограничения 32

30. План изучения и самопроверка 33

31. Словарь терминов проекта 34

32. Источники и границы объяснения 35

Приложение 01. JobBoardApplication.java 36

Приложение 02. Job.java 37

Приложение 03. JobRequest.java 38

Приложение 04. JobRepository.java 39

Приложение 05. JobService.java 40

Приложение 06. JobController.java 41

Приложение 07. JobNotFoundException.java 42

Приложение 08. ApiExceptionHandler.java 43

Приложение 09. application.properties 44

Приложение 10. pom.xml 45

Приложение 11. app.js 46

Приложение 12. JobApiTests.java 48

01. Что именно мы построили

Job Board - небольшое приложение для хранения вакансий. Пользователь вводит название роли, компанию, местоположение, описание и требуемый опыт. Приложение выдаёт вакансии списком, ищет по названию или компании, позволяет менять и удалять записи.

Это один Java-процесс. Внутри него работает встроенный веб-сервер Tomcat, Spring управляет Java-объектами, а H2 хранит данные. Браузер - отдельная программа, которая загружает HTML, CSS и JavaScript с этого сервера. H2 здесь не требует отдельного запущенного сервера базы.

Три разных вида данных

1. HTML описывает страницу: заголовок, поля формы, кнопки и контейнер вакансий.

2. JSON передаёт сведения о вакансиях между браузером и API.

3. Таблица jobs хранит записи в базе. Java-объект Job представляет такую запись в программе.

Вакансии не зашиты в исходники и не хранятся в JavaScript-массиве как единственном источнике. При загрузке страницы браузер запрашивает текущие записи у API. Если перезагрузить страницу, список снова приходит из базы.

Граница проекта

Нет регистрации, ролей, откликов кандидатов, загрузки файлов или публичного хостинга. У всех, кто имеет доступ к локальному приложению, одинаковые возможности. Это осознанный небольшой объём для изучения backend, а не готовая площадка найма.

> Как сказать на собеседовании: «Это приложение для управления вакансиями. Я могу показать полный путь HTTP-запроса, CRUD, валидацию и работу с базой через Spring Data JPA». Говори о своём понимании и реальном вкладе; не утверждай, что писал всё без помощи.

02. Карта файлов и порядок чтения

```
src/main/java/com/zinchenko/jobboard/
  JobBoardApplication.java
  job/
    Job.java
    JobRequest.java
    JobRepository.java
    JobService.java
    JobController.java
    JobNotFoundException.java
  common/ApiExceptionHandler.java
src/main/resources/
  application.properties
  static/index.html
  static/style.css
  static/app.js
src/test/java/com/zinchenko/jobboard/JobApiTests.java
```

src/main/java - код приложения. Название пакета начинается с com.zinchenko.jobboard; оно организует классы и предотвращает совпадения имён. Оно не регистрирует домен и ничего не публикует в интернет.

src/main/resources - файлы, которые попадут в classpath вместе с приложением. Spring Boot умеет раздавать файлы из static. application.properties содержит настройки сервера и базы. src/test/java используется при тестировании и не является кодом пользовательских функций.

pom.xml описывает сборку и зависимости. mvnw и mvnw.cmd запускают Maven Wrapper на macOS/Linux и Windows. target появляется при сборке. data появляется при работе файловой H2. .idea содержит локальные настройки IntelliJ.

Как читать без перегрузки

Сначала открой Job, чтобы увидеть предметную область. Затем JobRequest, Controller, Service и Repository. После этого пройди браузерный сценарий в app.js, обработчик ошибок и тесты. Не нужно в первый день знать внутреннее устройство Hibernate: сначала пойми связи между собственными классами.

03. Что делает main и как стартует Spring

```
@SpringBootApplication
public class JobBoardApplication {
    public static void main(String[] args) {
        SpringApplication.run(JobBoardApplication.class, args);
    }
}
```

main - точка входа Java-приложения. public означает доступность метода, static позволяет вызвать его без создания экземпляра класса, void означает отсутствие возвращаемого результата. args содержит аргументы запуска: например, --server.port=8088.

SpringApplication.run запускает контекст Spring. Упрощённо это контейнер, который знает, какие объекты нужны приложению, создаёт их и связывает зависимости. Далее Spring Boot подбирает конфигурацию на основе зависимостей и настроек, создаёт веб-инфраструктуру, подключает JPA и запускает Tomcat.

@SpringBootApplication объединяет несколько возможностей: объявление конфигурации, автоматическую конфигурацию и сканирование компонентов. Сканирование начинается с пакета главного класса и его подпакетов. Поэтому job и common расположены внутри com.zinchenko.jobboard.

Что происходит при запуске

1. JVM загружает главный класс и вызывает main.

2. Spring читает application.properties и аргументы запуска.

3. Контейнер регистрирует контроллер, сервис, обработчик ошибок и инфраструктуру репозитория.

4. Hibernate сопоставляет Job с таблицей jobs и при необходимости обновляет схему.

5. Tomcat начинает слушать локальный порт. Теперь можно открывать браузер.

main не вызывается заново на каждую кнопку. Сервер остаётся работать, а каждый запрос передаётся в подходящий метод контроллера. Запустить main и скомпилировать проект - разные действия: ошибка компиляции возникает ещё до старта Spring.

04. Spring создаёт зависимости за нас

```
private final JobRepository repository;

public JobService(JobRepository repository) {
    this.repository = repository;
}
```

JobService нужен объект, который умеет читать и записывать вакансии. Вместо создания его через new внутри сервиса зависимость передаётся через конструктор. Это dependency injection, внедрение зависимости.

Spring видит @Service, регистрирует JobService как bean и ищет подходящий bean JobRepository для конструктора. Для интерфейса репозитория Spring Data создаёт реализацию во время запуска. Самостоятельно писать JobRepositoryImpl для стандартных CRUD-операций здесь не требуется.

У JobController аналогичная схема: он принимает JobService. Поле final указывает, что ссылка задаётся при создании объекта и затем не переназначается. final не делает сам объект автоматически неизменяемым или потокобезопасным.

Почему это удобно

Зависимости видны прямо в конструкторе. Класс нельзя создать без обязательной зависимости. Для изолированного unit-теста при необходимости можно передать подмену репозитория. Единственному конструктору этих Spring-компонентов не нужна дополнительная @Autowired.

Контроллер и сервис обычно являются singleton-bean в контейнере. Их экземпляры могут обслуживать разные запросы. Поэтому мы не храним в полях сервиса текущую вакансию или введённую пользователем форму. Эти данные существуют в аргументах и локальных переменных методов.

> Сервис не является «особым видом Java». Это обычный Java-класс. Аннотации позволяют Spring добавить управление жизненным циклом и инфраструктуру вокруг его методов.

05. Полный путь открытия страницы

Когда ты открываешь http://localhost:8088/, браузер сначала запрашивает страницу. Статический обработчик Spring Boot находит index.html. Отдельно браузер загружает /style.css и /app.js. Контроллер JobController не создаёт HTML: он обслуживает только /api/jobs.

```
GET /                -> static/index.html
GET /style.css       -> static/style.css
GET /app.js          -> static/app.js
GET /api/jobs?q=     -> JobController.list(...)
```

В HTML подключение script использует defer. Это позволяет выполнить app.js после разбора HTML, когда поля формы и контейнер списка уже существуют. В конце app.js вызывается load(), поэтому загрузка вакансий начинается автоматически.

load читает поле поиска, вызывает api, а api использует fetch. Ответом API служит JSON-массив. JavaScript превращает элементы массива в DOM-узлы и вставляет их в контейнер #jobs. До завершения запроса виден текст загрузки; при пустом списке - сообщение об отсутствии вакансий.

Почему localhost не является публичным сайтом

localhost обозначает твой компьютер. Порт 8088 указывает конкретное приложение на нём. Когда сервер остановлен, ссылка не запускает Java автоматически. На компьютере работодателя localhost будет указывать на его компьютер, а не на твой Mac.

GitHub хранит исходники. GitHub Pages не запускает такой Java backend. Для публичной демонстрации понадобятся отдельное развёртывание и решения по доступу и базе. В текущем задании приложение работает локально.

06. HTTP, JSON и адреса API

HTTP-запрос включает метод, адрес, заголовки и иногда тело. Метод показывает действие, адрес - ресурс. Например, GET /api/jobs/1 читает вакансию с ID 1, DELETE по тому же адресу удаляет её.

```
POST /api/jobs
Content-Type: application/json

{
  "title": "Junior Java Developer",
  "company": "Example Studio",
  "location": "Remote",
  "description": "Build and test Java services",
  "experienceYears": 0
}
```

JSON - текстовый формат обмена данными. Названия полей здесь соответствуют компонентам JobRequest. Это не Java-код и не строка SQL. Jackson, подключённый через Spring Web, преобразует JSON в Java-объект и обратно.

Контракт ответов

1. 200 OK - чтение списка/записи или успешное обновление. Пустой поиск возвращает [] и 200, это не ошибка.

2. 201 Created - запись создана. В теле есть объект с id, а заголовок Location указывает адрес новой записи.

3. 204 No Content - удаление прошло успешно, тела ответа нет.

4. 400 Bad Request - запрос некорректен: неверные поля, JSON или тип ID.

5. 404 Not Found - запрошенная вакансия отсутствует.

Параметр ?q=java - query parameter для поиска. Число в /api/jobs/1 - path variable. Тело POST/PUT - отдельная часть запроса. Эти три способа передачи значений нельзя считать одним и тем же.

07. DTO: что такое JobRequest

```
public record JobRequest(
    @NotBlank @Size(max = 100) String title,
    @NotBlank @Size(max = 100) String company,
    @NotBlank @Size(max = 100) String location,
    @NotBlank @Size(max = 4000) String description,
    @NotNull @Min(0) @Max(50) Integer experienceYears
) {}
```

DTO означает Data Transfer Object: объект для передачи данных через границу приложения. JobRequest описывает поля, которые клиенту разрешено отправлять при создании и обновлении. В нём нет id: ID создаёт база, а при обновлении он берётся из URL.

record - компактная форма Java-класса для набора значений. Java генерирует конструктор, методы title(), company() и другие, а также equals, hashCode и toString. Компоненты нельзя переназначать после создания записи. Здесь внутри String и Integer, поэтому нет изменяемой коллекции, требующей дополнительной защиты.

Integer выбран специально: он может иметь значение null. Благодаря @NotNull отсутствующий опыт можно отклонить. Если бы использовали примитивный int, в некоторых сценариях отсутствующее значение превратилось бы в 0, и отсутствие поля было бы сложнее отличить от корректного нулевого опыта.

Почему не принимать Job напрямую

Job отражает хранение данных, включая id и JPA-аннотации. DTO отражает внешний контракт ввода. Если позже у Job появится служебное поле, оно не начнёт автоматически приниматься из JSON. В текущей версии ответы пока возвращают сам Job: это упрощение, а не полное разделение всех входных и выходных моделей.

> DTO не хранится в таблице сам по себе. В сервисе из него создаётся Job или обновляются поля уже существующей сущности.

08. Валидация: где отсекаются ошибки

В форме есть required, maxlength, min и max. Они помогают человеку, но не защищают API: можно отправить запрос напрямую, минуя браузерную форму. Поэтому ограничения повторяются на сервере в JobRequest.

@NotBlank отклоняет null, пустую строку и строку из пробельных символов. @Size ограничивает длину текста. @Min(0) и @Max(50) ограничивают опыт, а @NotNull требует его присутствия. Эти аннотации описывают ограничения; @Valid в аргументе контроллера включает проверку DTO при обработке запроса.

```
public ResponseEntity<Job> create(
        @Valid @RequestBody JobRequest request) {
    Job job = service.create(request);
    ...
}
```

Сначала Spring читает тело запроса и преобразует JSON в JobRequest. Затем проверяет ограничения. При нарушении этого DTO-контракта обработка не доходит до service.create. Исключение получает обработчик ошибок и формирует ответ 400.

Пример ошибки

```
{
  "title": "   ",
  "company": "Example Studio",
  "location": "Remote",
  "description": "A real description",
  "experienceYears": -1
}
```

Здесь ошибочны title и experienceYears. Сервер не должен записать вакансию с такими значениями. В Job.update дополнительно используется strip(), чтобы убрать пробелы по краям корректного текста. strip не заменяет валидацию: самостоятельно он просто превращает " " в пустую строку.

Важное ограничение: эти ограничения автоматически проверяются на HTTP-входе, а не при любом ручном new JobRequest в Java-коде. Внутренний вызов сервиса из нового компонента тоже должен соблюдать контракт.

09. Entity: как Job превращается в запись

```
@Entity
@Table(name = "jobs")
public class Job {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String title;
}
```

@Entity сообщает JPA, что класс представляет сохраняемую сущность. @Table задаёт таблицу jobs. @Id отмечает первичный ключ, а IDENTITY поручает базе генерировать значение ID при вставке. Не следует обещать, что ID будут всегда идти без пропусков.

JPA - спецификация, Hibernate - её реализация в этом приложении, H2 - сама база. Это разные уровни. Hibernate использует JDBC для обмена SQL с базой; Spring Data предоставляет удобный интерфейс поверх этого.

У новой Job id равен null. После сохранения ORM получает сгенерированный ID и связывает его с объектом. Long используется вместо long, чтобы можно было представить отсутствие ID до сохранения.

Конструкторы и доступ к полям

protected Job() нужен JPA для создания объекта при чтении записи. public Job(JobRequest request) создаёт объект из входных данных и делегирует заполнение методу update. Поскольку @Id стоит на поле, JPA использует доступ к полям; наличие публичного сеттера на каждое поле не требуется.

@Column(nullable=false) задаёт требование к колонке. Это не то же самое, что @NotBlank: ограничение NOT NULL не запрещает строку из пробелов. Поэтому ограничения API и схемы решают разные задачи. getTitle и остальные геттеры позволяют читать значения; Jackson использует их при формировании JSON.

10. Что хранит база и как сопоставляются поля

Каждая вакансия - отдельная строка таблицы jobs. Для данной конфигурации имена camelCase полей преобразуются стандартной стратегией Hibernate/Spring Boot в имена колонок с подчёркиваниями, например experienceYears в experience_years.

```
jobs
-----------------------------------------------------
id | title                 | company        | ...
1  | Junior Java Developer | Example Studio | ...
2  | Backend Intern        | North Team     | ...
```

id идентифицирует запись, а title описывает её. Две вакансии могут иметь одинаковые названия и компании: уникальность этих полей не настроена. В базе нет автоматического бизнес-правила «одна вакансия на компанию». Если оно понадобится, его придётся сформулировать и реализовать отдельно.

experienceYears - целое число. Нулевой опыт допустим: это полезно для стажировок и junior-позиций. description имеет лимит 4000, остальные текстовые поля - 100. И API, и схема согласованы по этим пределам.

Состояние Java-объекта и состояние базы

Изменение поля обычного объекта само по себе не обязательно является записью в базу. Важны контекст JPA и транзакция. Сущность, загруженная в активном persistence context, отслеживается Hibernate. При завершении транзакции изменения могут быть отправлены в SQL. В сервисе также явно вызывается repository.save.

Пример SQL вроде INSERT INTO jobs (...) VALUES (...) помогает понять направление работы, но не нужно утверждать, что Hibernate выдаёт всегда ровно одну заранее известную строку SQL. Число и момент запросов зависят от операции и поведения ORM.

11. Repository: интерфейс без своей реализации

```
public interface JobRepository
        extends JpaRepository<Job, Long> {
    List<Job>
    findByTitleContainingIgnoreCaseOrCompanyContainingIgnoreCaseOrderByIdDesc(
        String title, String company);
}
```

JpaRepository<Job, Long> говорит, с какой сущностью работает репозиторий и какой тип у её ID. Интерфейс наследует готовые методы: findAll, findById, save, delete, count и другие. Реализацию создаёт инфраструктура Spring Data.

Длинное имя поиска - декларация запроса. findBy начинает условие, TitleContainingIgnoreCase означает «title содержит строку без учёта регистра», Or соединяет альтернативы, CompanyContainingIgnoreCase повторяет условие для company, OrderByIdDesc задаёт сортировку по ID по убыванию.

В сервисе одинаковый поисковый текст передаётся в оба аргумента: term, term. Поэтому Java найдёт либо соответствие в названии, либо в компании. Строка "java" найдёт название "Junior Java Developer". Проверка по location пока не выполняется.

Что стоит знать о поиске

Derived query с Containing обрабатывает специальные символы LIKE как буквальные значения. Тест с процентом проверяет, что запрос "%" не превращается в «найти всё». Мы не собираем SQL конкатенацией пользовательского текста.

Для небольшого списка это понятно и достаточно. В большом приложении поиск подстроки и отсутствие пагинации могут быть дорогими. Длинное имя метода также можно заменить явно описанным запросом, когда условия становятся сложнее. Сейчас такой сложности нет.

12. Service: зачем отдельный слой

JobService объединяет операции над вакансиями. Контроллер не должен сам решать, как находить запись, в какой транзакции её менять и какое исключение бросать при отсутствии. Сервис не знает о HTML-кнопках и не формирует ResponseEntity.

```
public Job findById(Long id) {
    return repository.findById(id)
        .orElseThrow(() -> new JobNotFoundException(id));
}
```

findById репозитория возвращает Optional<Job>: запись либо существует, либо отсутствует. Optional - не коллекция вакансий и не null. orElseThrow извлекает значение либо создаёт исключение. Лямбда () -> ... откладывает создание исключения до случая отсутствия записи.

Как работает findAll

Если query равен null или состоит из пробелов, сервис получает все вакансии с Sort.Direction.DESC по id. Условие query == null || query.isBlank() использует короткое замыкание: если первое верно, второй вызов не выполняется, поэтому NullPointerException не возникает.

Если поисковая строка содержательная, strip() удаляет пробелы с краёв. Затем вызывается метод поиска репозитория. Сортировка по ID удобна для показа недавно добавленных записей, но это не полноценное время публикации: createdAt в модели пока отсутствует.

Почему сервис не слишком сложный

Для одной сущности бизнес-логики действительно немного. Отдельный слой здесь показывает границу сценариев и размещает транзакции в одном месте. Не нужно придумывать интерфейс JobService плюс единственную реализацию или несколько абстрактных фабрик без реальной причины.

13. Создание: от кнопки до commit

1. Пользователь заполняет форму и нажимает Save vacancy. Браузер проверяет HTML-ограничения.

2. Обработчик form.onsubmit отменяет обычную отправку формы через preventDefault(). FormData собирает поля, а Number превращает опыт из строки в число.

3. api отправляет POST /api/jobs с JSON и Content-Type: application/json.

4. Spring выбирает JobController.create, Jackson создаёт DTO, @Valid проверяет его.

5. Контроллер вызывает service.create через управляемый Spring-объект. Начинается write-транзакция.

6. Сервис создаёт new Job(request). Метод update копирует и очищает текст.

7. repository.save сохраняет сущность. База генерирует id. Транзакция фиксируется при успешном выходе из сервисного вызова.

8. Контроллер возвращает 201, Location и тело вакансии. JavaScript очищает форму и заново загружает список.

```
@Transactional
public Job create(JobRequest request) {
    return repository.save(new Job(request));
}
```

Важно различать два обновления. Первое - запись в базу. Второе - перерисовка списка в браузере после повторного GET. Если создание удалось, а последующая загрузка списка временно не удалась, запись уже может находиться в базе. Поэтому при сетевой ошибке не стоит бездумно нажимать создание много раз.

Практика: открой инструменты разработчика браузера, вкладку Network, создай вакансию и найди сначала POST, затем GET. Посмотри payload, статус 201 и ответ с id. Это упражнение, а не дополнительная функция приложения.

14. Обновление и удаление по ID

```
@Transactional
public Job update(Long id, JobRequest request) {
    Job job = findById(id);
    job.update(request);
    return repository.save(job);
}

@Transactional
public void delete(Long id) {
    repository.delete(findById(id));
}
```

При Edit JavaScript запоминает editingId и заполняет форму значениями карточки. При сохранении отправляется PUT /api/jobs/{id}. Сервер сначала проверяет весь DTO, затем ищет существующую запись, меняет её поля и сохраняет. id не изменяется.

PUT в этом проекте означает замену всех редактируемых полей. Это не частичный PATCH: если не отправить обязательное location, запрос не должен молча оставить старое значение. Для частичных изменений понадобился бы отдельный контракт.

Для удаления клиент показывает confirm. После подтверждения посылает DELETE. Сервис сначала загружает запись через findById, поэтому отсутствующая вакансия даёт 404. Существующая удаляется, а контроллер возвращает 204 без JSON-тела. Клиент специально не вызывает response.json() для такого ответа.

Границы текущей реализации

Между загрузкой записи и записью возможны конкурентные изменения из другого клиента. В Job нет @Version, поэтому защита от потерянного обновления не реализована. Для локального учебного сценария это упрощение, которое полезно уметь назвать.

Повторный DELETE отсутствующего ID возвращает 404. Это не мешает понимать DELETE как идемпотентную операцию по итоговому состоянию: после одного или нескольких запросов запись отсутствует, даже если статусы ответов отличаются.

15. @Transactional без магических обещаний

Транзакция определяет границу согласованной работы с базой: успешно завершённые изменения фиксируются, а при соответствующем сбое могут быть отменены. В этом проекте граница сценария расположена на сервисе, чтобы чтение записи и её изменение относились к одной операции.

```
@Service
@Transactional(readOnly = true)
public class JobService {
    ...
    @Transactional
    public Job update(Long id, JobRequest request) { ... }
}
```

На уровне класса задан режим чтения по умолчанию. У create, update и delete есть собственная @Transactional без readOnly=true: они выполняются в обычной write-транзакции. readOnly - подсказка инфраструктуре, а не универсальная гарантия запрета любого SQL-записывания.

Управление работает через Spring proxy: внешний вызов контроллера проходит через обёртку сервиса. Она начинает транзакцию, вызывает метод и завершает транзакцию. По умолчанию RuntimeException или Error приводят к rollback; для checked exception нужны соответствующие настройки, если требуется откат.

Нюанс внутреннего вызова

update вызывает findById внутри того же объекта. Такой self-invocation не проходит повторно через proxy. Здесь это нормально: внешняя update уже открыла write-транзакцию, и findById работает внутри неё. Нельзя объяснять это как автоматическое переключение на readOnly-транзакцию при каждой строке вызова.

Hibernate отслеживает изменённые managed-сущности, поэтому в update явный save часто можно было бы убрать. Мы оставили его для понятности намерения. Даже успешный вызов save ещё не означает, что все ограничения базы уже проверены: некоторые ошибки проявляются при flush/commit.

16. Контроллер переводит HTTP в Java

```
@RestController
@RequestMapping("/api/jobs")
public class JobController {
    @GetMapping("/{id}")
    public Job get(@PathVariable Long id) {
        return service.findById(id);
    }
}
```

@RestController обозначает контроллер, возвращаемые значения методов которого записываются в тело HTTP-ответа. @RequestMapping задаёт общий префикс. @GetMapping, @PostMapping, @PutMapping и @DeleteMapping выбирают метод HTTP и путь.

@PathVariable Long id преобразует сегмент адреса в Long. Если адрес содержит not-a-number, преобразование не удаётся и получается 400, а не отсутствие вакансии. Для числа 999999 преобразование успешно, но запись может отсутствовать - тогда уже 404.

@RequestParam(required=false) String q читает поисковый параметр. При отсутствии параметра сервису может прийти null. @RequestBody читает JSON-тело запроса. @Valid проверяет его поля перед выполнением логики метода.

Зачем ResponseEntity

Для чтения достаточно вернуть Job или List<Job>: успешный ответ обычно будет 200. При создании нужно явно задать 201 и Location, а при удалении - 204 без тела. ResponseEntity позволяет управлять статусом, заголовками и телом вместе.

ResponseEntity<Void> означает, что ответ не содержит объект данных. URI.create формирует адрес ресурса; он не делает дополнительный HTTP-запрос. Возврат Java-объекта тоже не отдаёт клиенту сам объект JVM: Jackson сериализует его в JSON.

17. Ошибки и ApiExceptionHandler

Если запись не найдена, сервис бросает JobNotFoundException. Это RuntimeException с сообщением, содержащим ID. Без специально определённого обработчика исключение такого класса само по себе не описывало бы корректный HTTP-контракт 404.

```
@ExceptionHandler(JobNotFoundException.class)
public ProblemDetail notFound(JobNotFoundException exception) {
    return ProblemDetail.forStatusAndDetail(
        HttpStatus.NOT_FOUND, exception.getMessage());
}
```

@RestControllerAdvice регистрирует общий обработчик для REST-контроллеров. Унаследованный ResponseEntityExceptionHandler также обслуживает стандартные ошибки Spring MVC: например, некорректный JSON и ошибки преобразования параметров.

Для ошибок @Valid переопределён handleMethodArgumentNotValid. Он получает field errors, складывает сообщения в LinkedHashMap и добавляет карту в ProblemDetail как errors. putIfAbsent оставляет одно сообщение на поле, даже если нарушено несколько ограничений. LinkedHashMap сохраняет порядок добавления, но порядок получения самих ошибок не следует считать жёстким API-контрактом.

```
Пример структуры ответа, сообщения могут отличаться:
{
  "status": 400,
  "detail": "Check the submitted fields",
  "errors": { "title": "must not be blank" }
}
```

ProblemDetail даёт стандартную структуру описания проблемы. Не все ответы об ошибках содержат errors: например, ошибка JSON может иметь только общие поля. Неожиданные ошибки базы не превращены в красивое доменное сообщение для каждого возможного случая - это ограничение текущей версии.

18. H2 и application.properties

```
server.address=127.0.0.1
server.port=${PORT:8080}
spring.datasource.url=jdbc:h2:file:./data/jobboard
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
spring.h2.console.enabled=false
```

127.0.0.1 ограничивает адрес прослушивания локальным компьютером. ${PORT:8080} означает: взять переменную окружения PORT, если она задана, иначе использовать 8080. Аргумент запуска --server.port=8088 может явно изменить порт.

jdbc:h2:file:./data/jobboard выбирает H2 в файловом режиме. ./data отсчитывается от рабочей папки процесса, а не от того места, где написан файл application.properties. Если запустить JAR из другой папки, можно получить другую базу и подумать, что вакансии исчезли.

username=sa и пустой пароль - локальная учебная конфигурация. Консоль H2 отключена. Это не настройка для публичного размещения. Файл базы не должен попадать в Git вместе с личными данными.

Два важных параметра JPA

ddl-auto=update позволяет Hibernate попытаться привести схему к модели при запуске. Это удобно в демо, но не замена контролируемым миграциям. Нельзя обещать сохранность произвольных данных при любых будущих изменениях полей.

open-in-view=false не держит JPA EntityManager открытым на весь веб-запрос ради ленивой загрузки во время формирования ответа. Сейчас у Job только простые поля, поэтому их сериализация после сервиса не требует ленивых связей.

Почему H2

Её легко запустить на Mac вместе с Java-приложением. Для junior это уменьшает количество настроек, позволяя сосредоточиться на API и JPA. Это настоящая SQL-база, но успешные тесты H2 не доказывают полную совместимость с PostgreSQL: диалект, типы и поведение могут отличаться.

19. HTML и CSS: устройство интерфейса

index.html определяет структуру страницы. header содержит логотип Jobs и имя Zinchenko. main включает вступление, список вакансий и форму. footer - заключительную строку. Отдельная буква Z в логотипе заменена на Jobs.

У полей есть id и name. id связывает label с input и помогает находить элемент в DOM. name используется FormData для ключа отправляемого значения. Если переименовать name="company", но не изменить JavaScript и DTO, контракт сломается.

```
<label for="company">Company</label>
<input id="company" name="company"
       required maxlength="100">
```

Контейнер #jobs изначально пуст. Его заполняет JavaScript после ответа API. #message и #form-message показывают состояние операций. role="status" и aria-live помогают сообщать изменения пользователям вспомогательных технологий.

Как устроено оформление

style.css задаёт цвета, шрифты, отступы и карточки. Класс .layout использует CSS Grid: на широком экране список и форма стоят рядом. В @media(max-width:760px) сетка становится одноколоночной. Кнопки и поля имеют видимую подсветку фокуса.

CSS не решает, как сохранить вакансию. Замена текста логотипа или цвета карточки не требует изменения Java-сервиса. Но если приложение запускается из JAR, обновлённые ресурсы нужно включить в новую сборку и перезапустить сервер, иначе он продолжит отдавать старую версию.

20. JavaScript: запросы и безопасный вывод

```
async function api(path = '', options = {}) {
    const response = await fetch('/api/jobs' + path, options);
    if (!response.ok) { ... }
    return response.status === 204 ? null : response.json();
}
```

async-функция возвращает Promise. await ожидает завершения асинхронной операции в этой функции, не превращая браузер в заблокированную программу. fetch обычно не бросает исключение только из-за статуса 400 или 404, поэтому код отдельно проверяет response.ok.

При ошибке api пытается прочитать ProblemDetail. Если есть errors, она объединяет сообщения полей; иначе использует detail или общий текст. Затем throw new Error передаёт сбой в catch вызывающей функции. При 204 возвращает null без попытки разобрать отсутствующий JSON.

element создаёт DOM-узел и записывает текст через textContent. Если в описании вакансии будет строка <script>...</script>, она отобразится как текст, а не исполнится как HTML. Это снижает риск внедрения разметки именно в этом способе вывода; не означает полную безопасность всего приложения.

Загрузка списка

load увеличивает loadVersion и запоминает номер своего запроса. После ответа сравнивает номер с актуальным. Если пользователь успел запустить новый поиск, старый ответ не должен перерисовать список поверх нового. Сами сетевые запросы при этом не отменяются.

encodeURIComponent кодирует поисковый текст для URL. replaceChildren очищает прежние карточки, затем создаются новые элементы. Пустой массив показывает empty state. Сообщение сетевой ошибки отображается отдельно от обычного пустого результата.

21. JavaScript: форма, Edit и Delete

editingId изначально null. Это означает создание. Нажатие Edit присваивает ID выбранной вакансии и переносит значения карточки в форму. Заголовок меняется на Edit vacancy, становится доступна кнопка Cancel editing.

```
const payload = Object.fromEntries(new FormData(form));
payload.experienceYears = Number(payload.experienceYears);

await api(editingId === null ? '' : '/' + editingId, {
    method: editingId === null ? 'POST' : 'PUT',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify(payload)
});
```

FormData даёт значения полей в основном как строки. Опыт переводится в число до сериализации. JSON.stringify создаёт строку тела запроса. Тот же HTML-form обслуживает создание и обновление; меняются URL и HTTP-метод.

Пока сохранение идёт, save.disabled=true уменьшает риск повторного нажатия. finally снова включает кнопку даже при ошибке. reset очищает форму, editingId и режим редактирования. В коде используется form.reset(), то есть возврат полей к исходным значениям HTML; опыт снова становится 0.

Удаление

Delete спрашивает подтверждение. При успехе карточки загружаются заново. Если удалялась редактируемая вакансия, форма также сбрасывается. При ошибке пользователь видит сообщение, а кнопка снова включается.

Упрощение интерфейса: подтверждение браузерное, нет отдельных экранов деталей и страниц. Нет отмены удаления и корзины. Блокировка кнопки полезна, но не является серверной гарантией идемпотентного создания: backend не имеет ключа дедупликации POST.

22. Maven: что именно делает pom.xml

Maven управляет зависимостями и шагами сборки. pom.xml - описание проекта для Maven, а не исходный Java-класс. groupId=com.zinchenko, artifactId=job-board и version=1.0.0 задают координаты артефакта. packaging по умолчанию - jar.

Родитель spring-boot-starter-parent версии 3.5.16 управляет согласованными версиями многих зависимостей и плагинов. Поэтому у каждого starter и H2 не задана своя версия вручную. Это помогает избежать случайной смеси несовместимых библиотек.

Что дают зависимости

1. starter-web - Spring MVC, обработка HTTP, JSON и встроенный Tomcat.

2. starter-data-jpa - репозитории, Hibernate и инфраструктура доступа к данным.

3. starter-validation - реализация проверки Jakarta Validation.

4. H2 со scope runtime - драйвер и движок нужны во время работы; Java-код использует JPA-абстракции.

5. starter-test со scope test - библиотеки тестирования доступны тестам, но не добавляются как пользовательская функциональность приложения.

spring-boot-maven-plugin собирает исполняемый JAR с зависимостями и позволяет запускать spring-boot:run. maven-compiler-plugin компилирует Java и теперь явно получает release=21. Java 21 также указана через source и target для корректного распознавания инструментами.

```
./mvnw test          # скомпилировать и выполнить тесты
./mvnw clean verify  # очистить, собрать, проверить
./mvnw spring-boot:run
java -jar target/job-board-1.0.0.jar
```

Wrapper скачивает указанную в .mvn/wrapper конфигурации версию Maven. JDK он не устанавливает: Java 21 должна быть доступна отдельно. При первом запуске требуется интернет для Maven и зависимостей.

23. Ошибка release version 5: что произошло

Сообщение SDK 21 does not support source version 1.5 означает конфликт: установлен современный JDK, но компиляции передан старый уровень языка Java 5. Это не просьба установить Java 5. До запуска main дело ещё не дошло.

Ранее Maven уже успешно компилировал проект с Java 21. Следовательно, ошибка наблюдалась в пути сборки IntelliJ. Без полной диагностики её внутренней модели нельзя честно назвать единственный источник, но наиболее вероятны несинхронизированные настройки модуля или неполный импорт Maven.

```
<properties>
  <java.version>21</java.version>
  <maven.compiler.release>21</maven.compiler.release>
  <maven.compiler.source>21</maven.compiler.source>
  <maven.compiler.target>21</maven.compiler.target>
</properties>
```

В POM явно закреплена Java 21, добавлена конфигурация compiler plugin. Локальный compiler.xml IntelliJ также указывает bytecode target 21. Но уже открытая IDE может хранить старую модель в памяти: изменение файлов на диске само по себе не доказывает, что IDE перечитала их.

Если ошибка осталась в IntelliJ

1. Открой именно pom.xml этой папки как Maven-проект.

2. В окне Maven выполни Reload All Maven Projects и дождись окончания синхронизации.

3. Проверь Project SDK и Module SDK: 21. Language level должен соответствовать 21 или наследоваться из корректного проекта.

4. В настройках Java Compiler проверь target bytecode 21; в Maven importer/runner выбери JDK 21.

5. Повтори запуск. Сравни с ./mvnw clean verify, чтобы отделить ошибку IDE от ошибки кода.

Тестовый сайт сейчас использует порт 8088. При обычном запуске main без аргументов используется 8080. Если порт занят, это отдельная ошибка, не связанная с source version.

24. Как устроены интеграционные тесты

```
@SpringBootTest(properties = {
  "spring.datasource.url=jdbc:h2:mem:jobtests;DB_CLOSE_DELAY=-1",
  "spring.jpa.hibernate.ddl-auto=create-drop"
})
@AutoConfigureMockMvc
class JobApiTests { ... }
```

@SpringBootTest поднимает контекст приложения. Это не изолированный unit-тест одного метода: используются настоящие сервис, репозиторий и тестовая база. @AutoConfigureMockMvc подключает MockMvc для запросов через MVC-инфраструктуру без отдельного реального сетевого сервера.

jdbc:h2:mem:jobtests выбирает базу в памяти. Она отделена от локального data/jobboard. create-drop создаёт схему для тестового контекста и удаляет при его закрытии. DB_CLOSE_DELAY=-1 оставляет in-memory базу доступной при закрытии отдельных соединений в рамках процесса.

@BeforeEach вызывает repository.deleteAll перед каждым тестом. Поэтому один тест не должен зависеть от записей другого. При этом последовательность ID не обязана начинаться с 1 после удаления всех строк: тесты читают ID из фактического ответа создания.

Вспомогательный createJob

Метод посылает POST с JSON, проверяет 201 и наличие Location, проверяет очистку пробелов в title, затем извлекает id через ObjectMapper. Он не является отдельным @Test: его используют несколько тестовых сценариев.

MockMvc проверяет, как вход проходит через контроллер и обработчики. Он не запускает app.js, не нажимает реальные кнопки и не проверяет CSS. Поэтому успешный API-тест не равен доказательству идеального интерфейса. Создание и редактирование в настоящем браузере проверялись отдельно вручную.

25. Что проверяет каждый из семи тестов

1. createsReadsUpdatesAndDeletesJob

Создаёт запись, проверяет её наличие через репозиторий, читает API, меняет location на Remote, удаляет, проверяет отсутствие в базе и 404 при повторном чтении. Это основной сквозной CRUD-сценарий.

2. searchesCaseInsensitivelyByTitleAndCompany

Проверяет JAVA, north, пробелы вокруг Java и пустой запрос. Проверяет отсутствие результатов для Python и буквального %. Это тест логики поиска, а не только факта ответа 200.

3. rejectsInvalidInputWithoutWritingToDatabase

Передаёт пустое название, отрицательный опыт, null-опыт и компанию длиннее 100 символов. Ожидает 400 с errors и в конце подтверждает, что база пуста.

4. rejectsInvalidUpdateAndPreservesOriginal

Создаёт валидную запись, пытается обновить её с пустым location, ожидает 400, затем подтверждает старое Warsaw. Он показывает, что отклонённый ввод не изменил запись. Это не отдельное доказательство rollback при ошибке посередине SQL-транзакции: DTO отклоняется раньше сервиса.

5. reportsMissingJobsForAllOperations

Для отсутствующего ID проверяет чтение, обновление и удаление - везде ожидается 404.

6. returnsBadRequestForMalformedJsonAndId

Проверяет сломанный JSON и нечисловой ID. Ожидает 400, а не 500.

7. servesBrowserInterface

Проверяет выдачу index.html и наличие текста Zinchenko. Не проверяет размещение элементов, весь JavaScript и браузерные действия.

Чего нет: полный набор граничных значений каждого поля, конкурентные запросы, PostgreSQL, авторизация, автоматический браузерный набор. Семь успешных тестов дают конкретные гарантии, но не означают отсутствие всех возможных ошибок.

26. Как разбираться с ошибкой по шагам

Сначала определи слой. Ошибка java: ... до баннера Spring - компиляция. Баннер появился, но сервер не стартовал - конфигурация/запуск. Страница открылась, а операция не работает - исследуй HTTP-запрос, backend и браузер. Так ты не будешь менять базу ради ошибки CSS.

Практический порядок диагностики

1. Повтори проблему один раз и запиши конкретное действие: какой URL, какие поля, какое сообщение.

2. В браузере открой Network. Есть ли запрос? Какой метод и статус? Что отправлено в payload?

3. Посмотри response. 400 с errors указывает на контракт ввода, 404 - на адрес или отсутствующую запись, 500 - на неожиданный серверный сбой.

4. Проверь журнал Java. Читай первое содержательное Caused by, а не только последнюю строку стека.

5. Проверь сценарий напрямую через curl. Если API работает, а кнопка нет, ищи проблему в JavaScript или состоянии страницы.

6. После исправления повтори исходный сценарий и подходящие тесты. Не меняй несколько несвязанных вещей одновременно.

Поставь breakpoint

В IntelliJ можно остановиться на первой строке JobController.create, затем шагнуть в JobService.create и Job.update. Посмотри поля request и значение id до/после save. Делай это на тестовой записи. Если валидация отклонила запрос, breakpoint в теле create не сработает - это ожидаемо.

Пример: 400 при experienceYears=-1 не баг, а ожидаемая защита. Пустой список после запуска из другой рабочей папки может означать другую H2-базу. Address already in use означает занятый порт: выбери другой или останови собственный старый процесс.

27. Учебное изменение: фильтр по городу

Это упражнение пока не реализовано. Оно помогает показать, что ты можешь менять проект осмысленно. Цель: добавить необязательный точный фильтр location, сохранив поиск q по названию/компании.

Сначала сформулируй поведение

Реши, считать ли Warsaw и warsaw одним городом, обрезать ли пробелы, как работает Remote и что делать с пустой строкой. Выбери AND между фильтром города и поиском: компания или название должны подходить, и город должен подходить тоже.

Затем меняй слои по очереди

1. Добавь тест: две вакансии в разных городах, фильтр оставляет одну; комбинация q и location не возвращает лишнюю запись.

2. В Controller прими дополнительный @RequestParam(required=false) String location.

3. В Service нормализуй оба параметра и выбери понятный путь поиска.

4. В Repository опиши запрос с правильно сгруппированным OR и AND. Не составляй SQL из строк пользователя.

5. В интерфейсе добавь поле/выбор города и отправляй параметр только в согласованном формате.

6. Проверь пустые значения, регистр, комбинацию условий, сброс поиска и отсутствие результатов.

Не начинай с копирования огромного метода из интернета. Сначала запиши 3-4 примера входа и ожидаемых результатов. Если не можешь словами объяснить, какой список должен получиться, код ещё рано писать.

> После выполнения на собеседовании можно показать конкретный diff: новый параметр API, правило в сервисе, запрос репозитория, поле формы и тесты. Это убедительнее, чем просто утверждать «я знаю Spring».

28. Ответы на собеседовании: основные вопросы

«Расскажи путь запроса до базы»

«Форма отправляет JSON через fetch. Spring MVC выбирает метод JobController, Jackson собирает JobRequest, @Valid проверяет ограничения. Контроллер вызывает JobService, он в транзакции обращается к JobRepository. Spring Data и Hibernate выполняют работу с H2. После успешного сохранения контроллер отдаёт JSON и 201, а браузер обновляет список».

«Зачем DTO?»

«JobRequest определяет разрешённые входные поля и их валидацию. Он отделён от JPA-сущности, поэтому клиент не задаёт ID и не зависит от всех деталей хранения. Выход пока упрощён: возвращается Job; следующим шагом можно ввести response DTO».

«Почему отдельный сервис?»

«Контроллер отвечает за HTTP, а сервис - за сценарии операций и транзакции. Он одинаково решает поиск отсутствующей вакансии для чтения, изменения и удаления. В маленьком CRUD логики мало, но границы остаются понятными».

«Зачем @Transactional?»

«Она задаёт границу работы с базой вокруг сервисного вызова. На чтение стоит readOnly, на запись - обычная транзакция. Нужно учитывать proxy и правила rollback; это не просто аннотация, делающая любой код безопасным».

«Почему H2?»

«Чтобы запустить проект без отдельной установки БД. В приложении используется файл, в тестах - память. Если целевой сервер PostgreSQL, дополнительно нужны тесты и миграции под PostgreSQL, одной H2 недостаточно».

Не заучивай ответы как чужую речь. Открой упомянутые классы и покажи соответствующие строки. Если деталь не проверял, так и скажи, затем предложи способ проверки.

29. Вопросы глубже и честные ограничения

«Что происходит при неправильных данных?»

«Проверка @Valid срабатывает до вызова сервиса. ApiExceptionHandler возвращает ProblemDetail с 400 и errors. Сломанный JSON обрабатывается инфраструктурой MVC. Проверка HTML помогает пользователю, но сервер ей не доверяет».

«Что проверяют тесты?»

«Они поднимают Spring-контекст и отдельную H2, выполняют запросы через MockMvc, проверяют ответы и часть состояния базы. Это интеграционные тесты API, а не browser end-to-end. Отклонённый update проверяется повторным чтением старого значения».

«Почему нельзя просто выложить на сервер?»

«Нет аутентификации и разграничения прав; H2 и ddl-auto=update выбраны для локального демо. Перед публичным размещением нужны правила доступа, конфигурация окружения, контролируемая схема и решение по хранению данных».

«Есть ли защита от одновременных изменений?»

«Сейчас нет @Version и отдельного контроля потерянных обновлений. Последнее обновление может перезаписать предыдущее. Это одна из известных границ».

«Почему не микросервисы и React?»

«Размер задачи этого не требует. Один Spring Boot сервер и простой клиент позволяют показать backend-фундамент и не тратить время на инфраструктуру, которую пока нечем оправдать».

«Использовал ли ИИ?»

«Да, Codex помог с реализацией и документацией. Я изучаю код, могу показать, как работают слои, и отдельно обозначаю изменения, которые сделал сам». Говори так только после реального разбора. Не приписывай себе навыки, которые ещё не проверил на практике.

30. План изучения и самопроверка

Пять учебных подходов

1. Запуск и модель. Запусти сервер, создай запись, найди её поля в Job и JobRequest. Объясни разницу файла исходника и сохранённых данных.

2. HTTP и контроллер. Повтори GET/POST/PUT/DELETE через curl или клиент запросов. Для каждого назови URL, тело и статус.

3. Сервис и база. Поставь breakpoint, посмотри id и сохранение. Перезапусти приложение из той же папки и проверь данные.

4. Ошибки и тесты. Отправь неверный опыт, пустой title, отсутствующий ID. Найди тесты, которые подтверждают это поведение.

5. Собственное изменение. Выполни небольшое упражнение с фильтром или статусом. Сначала опиши ожидаемое поведение и добавь проверку.

Ты понимаешь основу, если можешь без подсказки

Показать вход запроса и DTO. Объяснить constructor injection. Найти место, где создаётся Job. Отличить JPA от Hibernate и H2. Объяснить пустой список 200 и отсутствующую запись 404. Показать, где генерируется ID. Рассказать, почему тест не трогает data/. Разобраться, почему Java 5 в настройках не равна установленному JDK 21.

Если пункт пока не получается, вернись к соответствующей главе и повтори действие руками. Для junior нормально не знать всё. Намного полезнее уметь рассуждать, проверять предположение и признавать предел знания, чем уверенно повторять неточное объяснение.

> Результат обучения - не количество прочитанных страниц. Результат - способность открыть этот проект, исправить небольшую ошибку, объяснить изменение и проверить его.

31. Словарь терминов проекта

API - Контракт взаимодействия с приложением: здесь HTTP-адреса /api/jobs, методы, JSON и статусы.

CRUD - Create, Read, Update, Delete: создание, чтение, изменение и удаление.

DTO - Объект передачи данных. JobRequest - входной контракт.

Entity - Сохраняемый объект, сопоставленный с таблицей. Здесь Job.

Bean - Объект, которым управляет контейнер Spring.

DI - Передача зависимостей объекту, здесь через конструктор.

ORM - Сопоставление объектов и реляционных таблиц; работу выполняет Hibernate.

JPA - Спецификация Java persistence, которую реализует Hibernate.

Repository - Интерфейс доступа к хранилищу.

Transaction - Граница согласованной операции с commit/rollback.

Commit - Фиксация изменений транзакции.

Rollback - Отмена изменений транзакции при соответствующем сбое.

Serialization - Преобразование Java-объекта в JSON; обратное - десериализация.

Classpath - Ресурсы и классы, доступные JVM приложению.

JAR - Архив Java-приложения. Исполняемый Boot JAR содержит приложение и зависимости.

MockMvc - Проверка Spring MVC через имитированные HTTP-запросы без реального сетевого сервера.

DOM - Объектное дерево HTML-страницы, которое изменяет JavaScript.

Port - Номер сетевого входа процесса; здесь 8080 по умолчанию, 8088 в демонстрации.

32. Источники и границы объяснения

Основной источник этого разбора - прочитанные локальные исходники текущего проекта: Java-классы, JobApiTests, pom.xml, application.properties и статические ресурсы. Примеры ошибок и объясняющие схемы иллюстрируют этот код; они не являются дополнительной скрытой функциональностью.

Технические детали сверены с официальной документацией. Веб-страницы могут открывать новую версию Spring по умолчанию; фактическая версия проекта зафиксирована в POM. Не обновляй зависимости только ради совпадения номера на странице документации.

Spring Boot 3.5: требования к среде
https://docs.spring.io/spring-boot/3.5/system-requirements.html

Spring Data JPA: query methods
https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html

Spring Framework: @Transactional
https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/annotations.html

Spring MVC: validation
https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-validation.html

Spring Framework: MockMvc
https://docs.spring.io/spring-framework/reference/testing/mockmvc.html

H2: возможности и режимы хранения
https://www.h2database.com/html/features.html

Maven Compiler: release
https://maven.apache.org/plugins/maven-compiler-plugin/examples/set-compiler-release.html

IntelliJ IDEA: Maven import
https://www.jetbrains.com/help/idea/maven-support.html

Далее приведены реальные файлы с номерами строк. Это приложение для чтения рядом с объяснениями, а не материал, который нужно заучивать. При дальнейших изменениях проекта номера строк и поведение могут измениться.

Приложение 01. JobBoardApplication.java

src/main/java/com/zinchenko/jobboard/JobBoardApplication.java

```
 1 | package com.zinchenko.jobboard;
 2 |
 3 | import org.springframework.boot.SpringApplication;
 4 | import org.springframework.boot.autoconfigure.SpringBootApplication;
 5 |
 6 | @SpringBootApplication
 7 | public class JobBoardApplication {
 8 |
 9 |     public static void main(String[] args) {
10 |         SpringApplication.run(JobBoardApplication.class, args);
11 |     }
12 |
13 | }
```

Приложение 02. Job.java

src/main/java/com/zinchenko/jobboard/job/Job.java

```
 1 | package com.zinchenko.jobboard.job;
 2 |
 3 | import jakarta.persistence.*;
 4 |
 5 | @Entity
 6 | @Table(name = "jobs")
 7 | public class Job {
 8 |     @Id
 9 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
10 |     private Long id;
11 |     @Column(nullable = false, length = 100)
12 |     private String title;
13 |     @Column(nullable = false, length = 100)
14 |     private String company;
15 |     @Column(nullable = false, length = 100)
16 |     private String location;
17 |     @Column(nullable = false, length = 4000)
18 |     private String description;
19 |     @Column(nullable = false)
20 |     private int experienceYears;
21 |
22 |     protected Job() {} // Required by JPA.
23 |
24 |     public Job(JobRequest request) { update(request); }
25 |
26 |     public void update(JobRequest request) {
27 |         title = request.title().strip();
28 |         company = request.company().strip();
29 |         location = request.location().strip();
30 |         description = request.description().strip();
31 |         experienceYears = request.experienceYears();
32 |     }
33 |
34 |     public Long getId() { return id; }
35 |     public String getTitle() { return title; }
36 |     public String getCompany() { return company; }
37 |     public String getLocation() { return location; }
38 |     public String getDescription() { return description; }
39 |     public int getExperienceYears() { return experienceYears; }
40 | }
```

Приложение 03. JobRequest.java

src/main/java/com/zinchenko/jobboard/job/JobRequest.java

```
 1 | package com.zinchenko.jobboard.job;
 2 |
 3 | import jakarta.validation.constraints.*;
 4 |
 5 | public record JobRequest(
 6 |         @NotBlank @Size(max = 100) String title,
 7 |         @NotBlank @Size(max = 100) String company,
 8 |         @NotBlank @Size(max = 100) String location,
 9 |         @NotBlank @Size(max = 4000) String description,
10 |         @NotNull @Min(0) @Max(50) Integer experienceYears) {
11 | }
```

Приложение 04. JobRepository.java

src/main/java/com/zinchenko/jobboard/job/JobRepository.java

```
1 | package com.zinchenko.jobboard.job;
2 |
3 | import org.springframework.data.jpa.repository.JpaRepository;
4 | import java.util.List;
5 |
6 | public interface JobRepository extends JpaRepository<Job, Long> {
7 |     List<Job>
  | findByTitleContainingIgnoreCaseOrCompanyContainingIgnoreCaseOrderByIdDesc(
8 |             String title, String company);
9 | }
```

Приложение 05. JobService.java

src/main/java/com/zinchenko/jobboard/job/JobService.java

```
 1 | package com.zinchenko.jobboard.job;
 2 |
 3 | import org.springframework.data.domain.Sort;
 4 | import org.springframework.stereotype.Service;
 5 | import org.springframework.transaction.annotation.Transactional;
 6 | import java.util.List;
 7 |
 8 | @Service
 9 | @Transactional(readOnly = true)
10 | public class JobService {
11 |     private final JobRepository repository;
12 |
13 |     public JobService(JobRepository repository) { this.repository = repository; }
14 |
15 |     public List<Job> findAll(String query) {
16 |         if (query == null || query.isBlank()) {
17 |             return repository.findAll(Sort.by(Sort.Direction.DESC, "id"));
18 |         }
19 |         String term = query.strip();
20 |         return repository.findByTitleContainingIgnoreCaseOrCompanyContainingIgnoreCas
   | eOrderByIdDesc(term, term);
21 |     }
22 |
23 |     public Job findById(Long id) {
24 |         return repository.findById(id).orElseThrow(() -> new
   | JobNotFoundException(id));
25 |     }
26 |
27 |     @Transactional
28 |     public Job create(JobRequest request) { return repository.save(new Job(request));
   |  }
29 |
30 |     @Transactional
31 |     public Job update(Long id, JobRequest request) {
32 |         Job job = findById(id);
33 |         job.update(request);
34 |         return repository.save(job);
35 |     }
36 |
37 |     @Transactional
38 |     public void delete(Long id) { repository.delete(findById(id)); }
39 | }
```

Приложение 06. JobController.java

src/main/java/com/zinchenko/jobboard/job/JobController.java

```
 1 | package com.zinchenko.jobboard.job;
 2 |
 3 | import jakarta.validation.Valid;
 4 | import org.springframework.http.ResponseEntity;
 5 | import org.springframework.web.bind.annotation.*;
 6 | import java.net.URI;
 7 | import java.util.List;
 8 |
 9 | @RestController
10 | @RequestMapping("/api/jobs")
11 | public class JobController {
12 |     private final JobService service;
13 |
14 |     public JobController(JobService service) { this.service = service; }
15 |
16 |     @GetMapping
17 |     public List<Job> list(@RequestParam(required = false) String q) { return
   | service.findAll(q); }
18 |
19 |     @GetMapping("/{id}")
20 |     public Job get(@PathVariable Long id) { return service.findById(id); }
21 |
22 |     @PostMapping
23 |     public ResponseEntity<Job> create(@Valid @RequestBody JobRequest request) {
24 |         Job job = service.create(request);
25 |         return ResponseEntity.created(URI.create("/api/jobs/" +
   | job.getId())).body(job);
26 |     }
27 |
28 |     @PutMapping("/{id}")
29 |     public Job update(@PathVariable Long id, @Valid @RequestBody JobRequest request)
   | {
30 |         return service.update(id, request);
31 |     }
32 |
33 |     @DeleteMapping("/{id}")
34 |     public ResponseEntity<Void> delete(@PathVariable Long id) {
35 |         service.delete(id);
36 |         return ResponseEntity.noContent().build();
37 |     }
38 | }
```

Приложение 07. JobNotFoundException.java

src/main/java/com/zinchenko/jobboard/job/JobNotFoundException.java

```
1 | package com.zinchenko.jobboard.job;
2 |
3 | public class JobNotFoundException extends RuntimeException {
4 |     public JobNotFoundException(Long id) { super("Job with id " + id + " was not
  | found"); }
5 | }
```

Приложение 08. ApiExceptionHandler.java

src/main/java/com/zinchenko/jobboard/common/ApiExceptionHandler.java

```
 1 | package com.zinchenko.jobboard.common;
 2 |
 3 | import com.zinchenko.jobboard.job.JobNotFoundException;
 4 | import org.springframework.http.*;
 5 | import org.springframework.web.bind.MethodArgumentNotValidException;
 6 | import org.springframework.web.bind.annotation.*;
 7 | import
   | org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;
 8 | import org.springframework.web.context.request.WebRequest;
 9 | import java.util.LinkedHashMap;
10 | import java.util.Map;
11 |
12 | @RestControllerAdvice
13 | public class ApiExceptionHandler extends ResponseEntityExceptionHandler {
14 |     @ExceptionHandler(JobNotFoundException.class)
15 |     public ProblemDetail notFound(JobNotFoundException exception) {
16 |         return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND,
   | exception.getMessage());
17 |     }
18 |
19 |     @Override
20 |     protected ResponseEntity<Object> handleMethodArgumentNotValid(
21 |             MethodArgumentNotValidException exception, HttpHeaders headers,
22 |             HttpStatusCode status, WebRequest request) {
23 |         Map<String, String> errors = new LinkedHashMap<>();
24 |         exception.getBindingResult().getFieldErrors().forEach(error ->
25 |                 errors.putIfAbsent(error.getField(), error.getDefaultMessage()));
26 |         ProblemDetail problem = ProblemDetail.forStatusAndDetail(status, "Check the
   | submitted fields");
27 |         problem.setProperty("errors", errors);
28 |         return handleExceptionInternal(exception, problem, headers, status, request);
29 |     }
30 | }
```

Приложение 09. application.properties

src/main/resources/application.properties

```
spring.application.name=zinchenko-job-board
server.address=127.0.0.1
server.port=${PORT:8080}
spring.datasource.url=jdbc:h2:file:./data/jobboard
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
spring.h2.console.enabled=false
```

Приложение 10. pom.xml

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
parent</artifactId><version>3.5.16</version><relativePath/></parent>
  <groupId>com.zinchenko</groupId><artifactId>job-
board</artifactId><version>1.0.0</version>
  <name>Zinchenko Job Board</name><description>A junior Java portfolio application
for managing job vacancies</description>
  <properties>
    <java.version>21</java.version>
    <maven.compiler.release>21</maven.compiler.release>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-
starter-web</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-
starter-data-jpa</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-
starter-validation</artifactId></dependency>
    <dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId><scope>ru
ntime</scope></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-
starter-test</artifactId><scope>test</scope></dependency>
  </dependencies>
  <build><plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <configuration>
        <release>${java.version}</release>
        <parameters>true</parameters>
      </configuration>
    </plugin><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-
boot-maven-plugin</artifactId></plugin></plugins></build>
</project>
```

Приложение 11. app.js

src/main/resources/static/app.js

```
const form = document.querySelector('#job-form');
const message = document.querySelector('#message');
const formMessage = document.querySelector('#form-message');
const save = document.querySelector('#save');
let editingId = null;
let loadVersion = 0;

async function api(path = '', options = {}) {
    const response = await fetch('/api/jobs' + path, options);
    if (!response.ok) {
        const problem = await response.json().catch(() => ({}));
        const details = problem.errors ? Object.entries(problem.errors).map(([field, error]) => `${field}: ${error}`).join('; ') : problem.detail;
        throw new Error(details || `Request failed (${response.status})`);
    }
    return response.status === 204 ? null : response.json();
}

function element(tag, text, className) {
    const node = document.createElement(tag);
    node.textContent = text;
    if (className) node.className = className;
    return node;
}

async function load() {
    const version = ++loadVersion;
    message.textContent = 'Loading vacancies…';
    try {
        const jobs = await api('?q=' + encodeURIComponent(document.querySelector('#query').value));
        if (version !== loadVersion) return;
        const container = document.querySelector('#jobs');
        container.replaceChildren();
        document.querySelector('#count').textContent = jobs.length;
        if (!jobs.length) container.append(element('p', 'No vacancies found. Add your first opportunity or change the search.', 'empty'));
        for (const job of jobs) {
            const card = element('article', '', 'card');
            card.append(element('h3', job.title), element('p', `${job.company} · ${job.location} · ${job.experienceYears} years experience`, 'meta'), element('p', job.description, 'description'));
            const actions = element('div', '', 'actions');
            const edit = element('button', 'Edit');
            edit.onclick = () => {

                editingId = job.id;
                for (const key of ['title', 'company', 'location', 'description', 'experienceYears']) form.elements[key].value = job[key];
                document.querySelector('#form-title').textContent = 'Edit vacancy';
                document.querySelector('#cancel').hidden = false;
                formMessage.textContent = '';
                form.elements.title.focus();
            };
            const remove = element('button', 'Delete', 'danger');
            remove.onclick = async () => {
                if (!confirm(`Delete "${job.title}"?`)) return;
                remove.disabled = true;
                try {
```

```
53 |                     await api('/' + job.id, {method: 'DELETE'});
54 |                     if (editingId === job.id) reset();
55 |                     await load();
56 |                 } catch (error) { message.textContent = error.message; }
57 |                 finally { remove.disabled = false; }
58 |             };
59 |             actions.append(edit, remove); card.append(actions);
   | container.append(card);
60 |         }
61 |         message.textContent = '';
62 |     } catch (error) { if (version === loadVersion) message.textContent =
   | error.message; }
63 | }
64 |
65 | function reset() {
66 |     editingId = null;
67 |     form.reset();
68 |     document.querySelector('#form-title').textContent = 'Add a vacancy';
69 |     document.querySelector('#cancel').hidden = true;
70 |     formMessage.textContent = '';
71 | }
72 | document.querySelector('#cancel').onclick = reset;
73 | document.querySelector('#search').onsubmit = event => { event.preventDefault();
   | load(); };
74 | form.onsubmit = async event => {
75 |     event.preventDefault();
76 |     const payload = Object.fromEntries(new FormData(form));
77 |     payload.experienceYears = Number(payload.experienceYears);
78 |     save.disabled = true;
79 |     formMessage.textContent = 'Saving…';
80 |     try {
81 |         await api(editingId === null ? '' : '/' + editingId, {

82 |             method: editingId === null ? 'POST' : 'PUT',
83 |             headers: {'Content-Type': 'application/json'}, body:
   | JSON.stringify(payload)
84 |         });
85 |         reset(); formMessage.textContent = 'Vacancy saved.'; await load();
86 |     } catch (error) { formMessage.textContent = error.message; }
87 |     finally { save.disabled = false; }
88 | };
89 | load();
```

Приложение 12. JobApiTests.java

src/test/java/com/zinchenko/jobboard/JobApiTests.java

```
 1 | package com.zinchenko.jobboard;
 2 |
 3 | import com.zinchenko.jobboard.job.JobRepository;
 4 | import org.junit.jupiter.api.BeforeEach;
 5 | import org.junit.jupiter.api.Test;
 6 | import org.springframework.beans.factory.annotation.Autowired;
 7 | import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
 8 | import org.springframework.boot.test.context.SpringBootTest;
 9 | import org.springframework.test.web.servlet.MockMvc;
10 | import org.springframework.http.MediaType;
11 | import com.fasterxml.jackson.databind.ObjectMapper;
12 | import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
13 | import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
14 | import static org.assertj.core.api.Assertions.assertThat;
15 |
16 | @SpringBootTest(properties =
   | {"spring.datasource.url=jdbc:h2:mem:jobtests;DB_CLOSE_DELAY=-1",
   | "spring.jpa.hibernate.ddl-auto=create-drop"})
17 | @AutoConfigureMockMvc
18 | class JobApiTests {
19 |     @Autowired MockMvc mvc;
20 |     @Autowired JobRepository repository;
21 |     @Autowired ObjectMapper mapper;
22 |
23 |     private static final String JOB = """
24 |         {"title":" Junior Java Developer ","company":"North
   | Studio","location":"Warsaw",
25 |          "description":"Build useful Java services","experienceYears":0}
26 |         """;
27 |
28 |     @BeforeEach void clearDatabase() { repository.deleteAll(); }
29 |
30 |     private long createJob() throws Exception {
31 |         String response =
   | mvc.perform(post("/api/jobs").contentType(MediaType.APPLICATION_JSON).content(JOB))
32 |
   | .andExpect(status().isCreated()).andExpect(header().exists("Location"))
33 |                 .andExpect(jsonPath("$.title").value("Junior Java Developer"))
34 |                 .andReturn().getResponse().getContentAsString();
35 |         return mapper.readTree(response).get("id").asLong();
36 |     }
37 |
38 |     @Test void createsReadsUpdatesAndDeletesJob() throws Exception {
39 |         long id = createJob();
40 |         assertThat(repository.findById(id)).isPresent();

41 |         mvc.perform(get("/api/jobs/" + id)).andExpect(status().isOk());
42 |         mvc.perform(put("/api/jobs/" + id).contentType(MediaType.APPLICATION_JSON)
43 |                 .content(JOB.replace("Warsaw", "Remote")))
44 |
   | .andExpect(status().isOk()).andExpect(jsonPath("$.location").value("Remote"));
45 |         mvc.perform(delete("/api/jobs/" + id)).andExpect(status().isNoContent());
46 |         assertThat(repository.findById(id)).isEmpty();
47 |         mvc.perform(get("/api/jobs/" + id)).andExpect(status().isNotFound());
48 |     }
49 |
50 |     @Test void searchesCaseInsensitivelyByTitleAndCompany() throws Exception {
51 |         createJob();
52 |         for (String query : new String[]{"JAVA", "north", "  Java  ", ""}) {
```

```
53 |             mvc.perform(get("/api/jobs").param("q", query))
54 |
   | .andExpect(status().isOk()).andExpect(jsonPath("$.length()").value(1));
55 |         }
56 |         mvc.perform(get("/api/jobs").param("q",
   | "Python")).andExpect(jsonPath("$.length()").value(0));
57 |         mvc.perform(get("/api/jobs").param("q",
   | "%")).andExpect(jsonPath("$.length()").value(0));
58 |     }
59 |
60 |     @Test void rejectsInvalidInputWithoutWritingToDatabase() throws Exception {
61 |         for (String invalid : new String[]{JOB.replace(" Junior Java Developer ", "
   | "),
62 |                 JOB.replace(":0", ":-1"),
63 |                 JOB.replace(":0", ":null"),
64 |                 JOB.replace("North Studio", "x".repeat(101))}) {
65 |             mvc.perform(post("/api/jobs").contentType(MediaType.APPLICATION_JSON).con
   | tent(invalid))
66 |
   | .andExpect(status().isBadRequest()).andExpect(jsonPath("$.errors").exists());
67 |         }
68 |         assertThat(repository.count()).isZero();
69 |     }
70 |
71 |     @Test void rejectsInvalidUpdateAndPreservesOriginal() throws Exception {
72 |         long id = createJob();
73 |         mvc.perform(put("/api/jobs/" + id).contentType(MediaType.APPLICATION_JSON)
74 |                 .content(JOB.replace("Warsaw", "
   | "))).andExpect(status().isBadRequest());
75 |         mvc.perform(get("/api/jobs/" +
   | id)).andExpect(jsonPath("$.location").value("Warsaw"));
76 |     }

77 |
78 |     @Test void reportsMissingJobsForAllOperations() throws Exception {
79 |         mvc.perform(get("/api/jobs/999999")).andExpect(status().isNotFound());
80 |         mvc.perform(put("/api/jobs/999999").contentType(MediaType.APPLICATION_JSON).c
   | ontent(JOB)).andExpect(status().isNotFound());
81 |         mvc.perform(delete("/api/jobs/999999")).andExpect(status().isNotFound());
82 |     }
83 |
84 |     @Test void returnsBadRequestForMalformedJsonAndId() throws Exception {
85 |
   | mvc.perform(post("/api/jobs").contentType(MediaType.APPLICATION_JSON).content("{"))
86 |                 .andExpect(status().isBadRequest());
87 |         mvc.perform(get("/api/jobs/not-a-
   | number")).andExpect(status().isBadRequest());
88 |     }
89 |
90 |     @Test void servesBrowserInterface() throws Exception {
91 |         mvc.perform(get("/index.html")).andExpect(status().isOk())
92 |
   | .andExpect(content().string(org.hamcrest.Matchers.containsString("Zinchenko")));
93 |     }
94 | }
```